

PROGRAMACIÓN CONCURRENTE CON SEMÁFOROS Y MONITORES

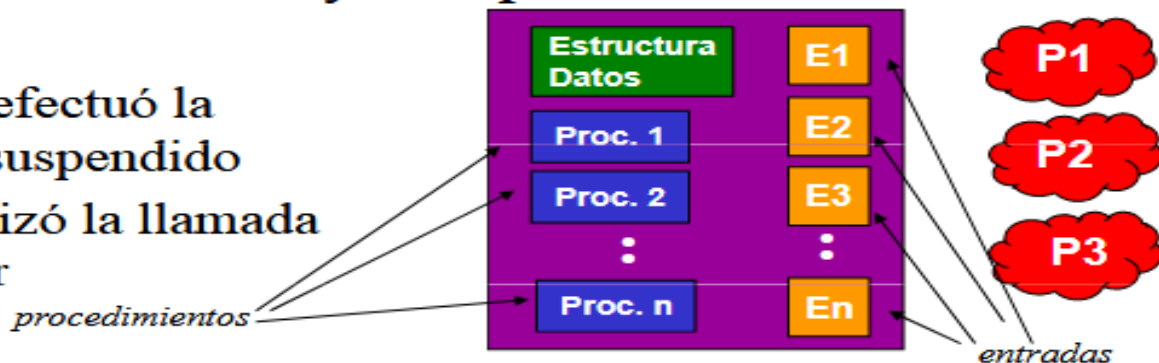
Sistemas Operativos

Prof. PhD Mirella Herrera

Monitores

- Procesos no tienen acceso directo a las estructuras de datos.
- Solo un proceso puede estar activo en un monitor en cada momento.
- Cuando se llama a un entrada la implementación monitor verifica si no hay otro proceso dentro del monitor:

- *si*: proceso efectuó la llamada es suspendido
- *no*: el que hizo la llamada puede entrar



Monitores

- Para sincronizar procesos se usan variables de tipo condición y dos operaciones:

1. wait()

- proceso descubre que no puede continuar (almacen lleno) ejecuta *wait* en alguna variable de condición proceso se bloquea y permite que otro proceso entre al monitor

2. signal

- proceso que entra puede despertar a otro con una instrucción *signal* sobre la variable de condición que el otro proceso espera

Monitores

- Las variables de condición NO son contadores.
- No se acumulan señales para su uso posterior (como los semáforos)
- Si una variable de condición queda señalada sin que nadie la espere, la señal se pierde.
- La instrucción WAIT debe aparecer antes de SIGNAL
 - regla que hace más sencilla la implementación

Monitores

- Exclusión mutua es automática,
 - el monitor garantiza que solo el productor o el consumidor estaran ejecutandose
- Si el almacen esta totalmente ocupado, el productor realizará una operación WAIT
 - se encontrará bloqueado hasta que consumidor ejecute un SIGNAL
- Si el almacen esta totalmente vacio el consumidor realizará una operación WAIT
 - se encontrará bloqueado hasta que productor ejecute un SIGNAL

Monitores

monitor ProdCons

```
condition lleno, vacio;  
integer cont, N;
```

```
procedure producir;  
begin  
    if (cont = N) then  
        wait(lleno);  
    introducir_elemento;  
    cont := cont + 1;  
    if (cont = 1) then  
        signal(vacio);  
end;
```

```
procedure retirar;  
begin  
    if (cont = 0) then  
        wait(vacio)  
    retirar_elemento;  
    cont := cont - 1  
    if (cont = N - 1) then  
        signal(lleno);  
end;
```

```
cont := 0; N = 100;  
end monitor;
```

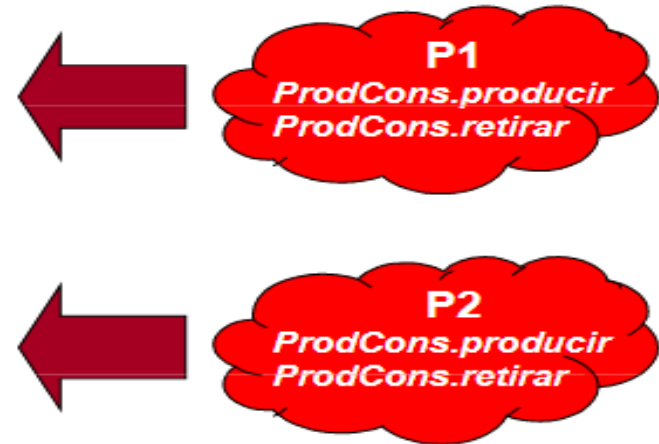
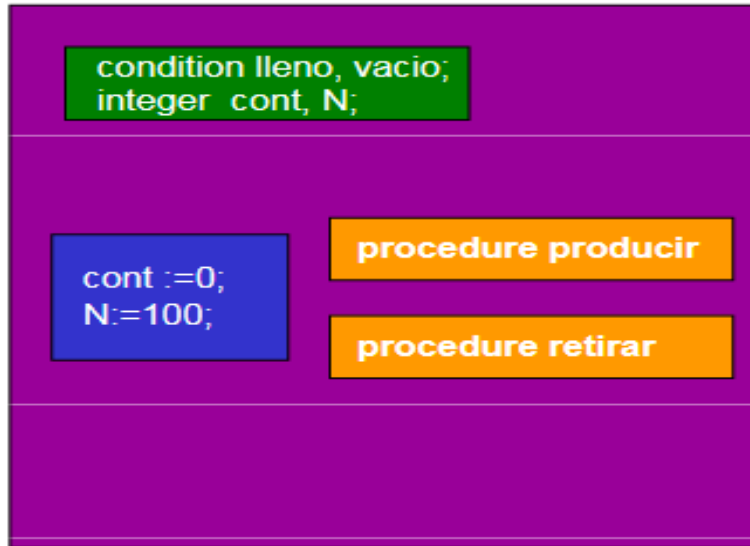
Monitores

```
procedure productor;  
begin  
  while true do  
  begin  
    producir_elemento;  
    ProdCons.producir;  
  end  
end;
```

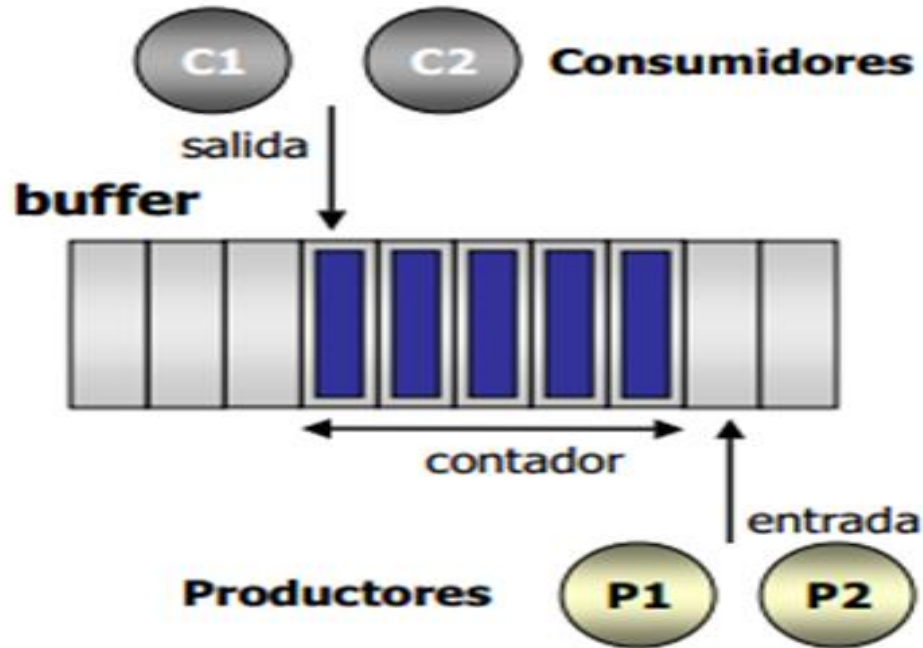
```
procedure consumidor;  
begin  
  while true do  
  begin  
    ProdCons.retirar;  
    consumir_elemento;  
  end  
end;
```

Monitores

monitor ProdCons



El problema: Productor-Consumidor



El problema: Productor-Consumidor

```
int contador = 0; //indica número de items en buffer
char buffer[N];
int in = 0; int out = 0;
sem mutex=1; sem vacio = N; sem lleno = 0;
```

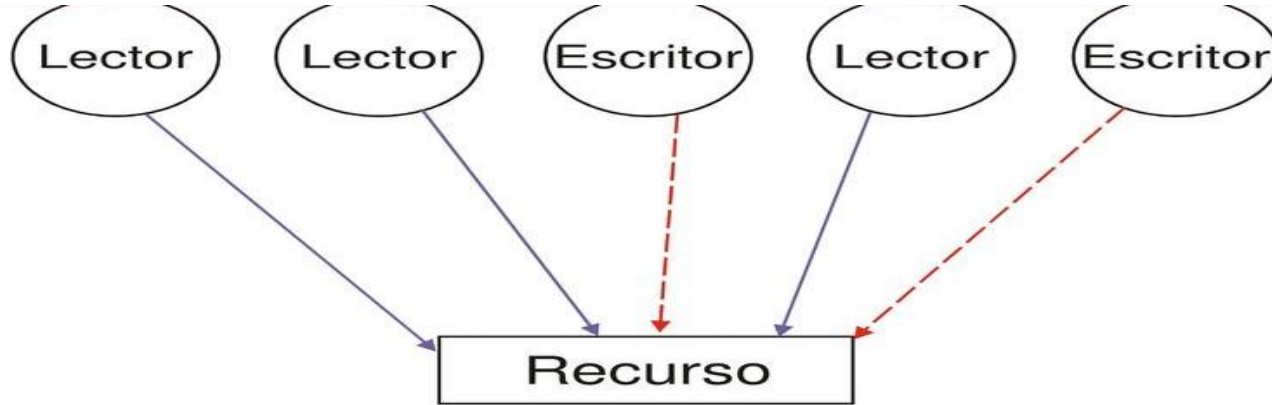
Productor

```
while (true) {
    /* produce un item en proxProd */
    wait(vacio);
    wait(mutex);
    buffer[in] = proxProd;
    in = (in + 1) % N;
    contador++;
    signal(mutex);
    signal(lleno);
}
```

Consumidor

```
While(true){
    wait(lleno);
    wait(mutex);
    proxCons = buffer[out];
    out = (out + 1) % N;
    contador--;
    signal(mutex);
    signal(vacio);
    /* consume proxCons */
}
```

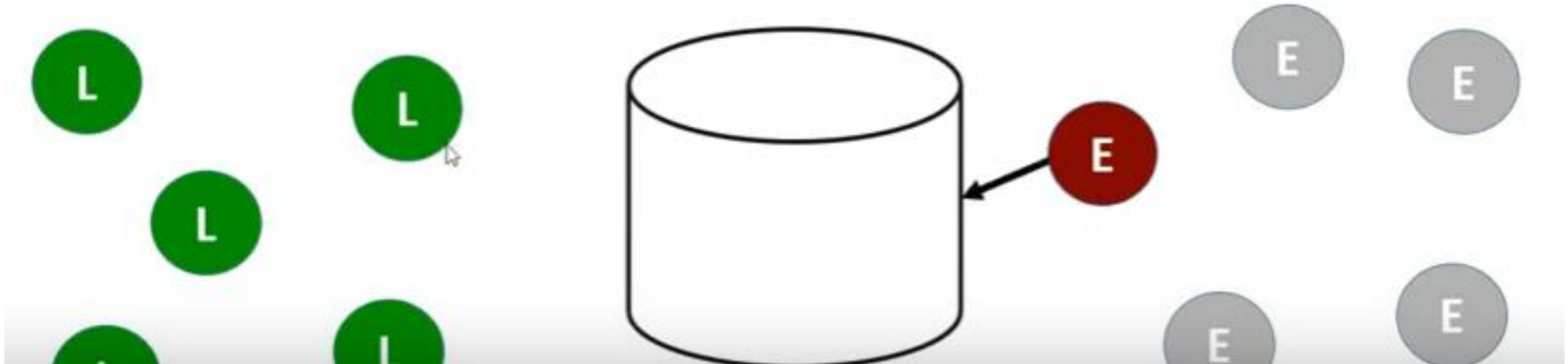
El problema: Lectores-Escritores



- Se da una combinación de procesos que leen y escriben sobre el recurso. Se dan las siguientes restricciones:
 - Sólo se permite acceso a un escritor al mismo tiempo. Ningún proceso lector o escritor puede acceder.
 - Se permite, que múltiples lectores tengan acceso al objeto ya que estos no modifican el contenido.

El problema: Lectores-Escritores con prioridad de los Escritores

No permitir acceder a los datos a ningún nuevo lector una vez que, un escritor haya declarado su deseo de escribir.



El problema: Lectores-Escritores con prioridad de los Escritores

```
int contlect=0,contesc=0;
Semaphore x,y,z,esem,lsem;
Lector()
{
    while(forever)
    {
        wait(z);
        wait(lsem);
        wait(x);
        contlect++;
        if(contlect==1) wait(esem);
        signal(x);
        signal(lsem);
        signal(z);
        LEER_UNIDAD;
        wait(x);
        contlect--;
        if(contlect==0) signal(esem);
        signal(x);
    }
}
```

```
Escritor()
{
    while(forever)
    {
        wait(y);
        contesc++;
        if(contesc==1) wait(lsem);
        signal(y);
        wait(esem);
        ESCRIBIR_UNIDAD;
        signal(esem);
        wait(y);
        contesc--;
        if(contesc==0) signal(lsem);
        signal(y);
    }
}
```

```
main()
{
    initsem(x,1);initsem(y,1);initsem(z,1);
    initsem(esem,1);initsem(lsem,1);
    cobegin {
        Lector(); Escritor();
    }
}
```

